Universidad de Huelva	**Examen de Metodología de la Programación** **Curso 2023-2024, Convocatoria I (Enero)**

Sección Informativa: **Duración del examen: 2 horas 30 minutos**

1- (3.50 Puntos). C++. Dada la clase Fecha, la clase Documento y su clase hija Libro:

```
class Fecha {
  int dia, mes, anio;
public:
  ...



};
```

```
class Documento {
  ...
  int idDocumento;
  char *titulo;
  char *autor;
public:
  ...
};
```

```
class Libro: public Documento {
  static double IVA;
  Fecha fechaEdicion;
  char *editorial;
  double precio;
public:
  ...
};
```

```
int main() {
  const Fecha f1(29,12,2024), f2(30,10,2001); //Fecha f4; //no permitido
  Fecha f3(1,1,2000); f3.setFecha(15,11,2000);
  cout << f1.getDia() << "-" << f1.getMes() << "-" << f1.getAnio() << endl;
  cout << f1 << ", " << f2 << ", " << f3 << endl;
  Documento d1("La Odisea", "Homero"), d2(d1);
  cout << d1 << endl << d2 << endl;
  if (d1==d2) cout << "d1 y d2 son exactamente iguales" << endl;
  else cout << "d1 y d2 no son exactamente iguales" << endl;
  d2=d1;
  cout << d1 << endl << d2 << endl;
  if (d1==d2)
    cout << "d1 y d2 son exactamente iguales" << endl;
  cout << "--LIBROS--" << endl;
  Libro::setIVA(0.27); //el IVA debe ser como maximo del 21%
  Libro i1("Genesis", "Pedro Salinas", f1, "Planeta", 10.00);
  Libro i2("Perdidos", "Juan Rulfo", f2, "Alfaguara", 12.75);
  Libro i3(i2);
  cout << i1 << endl << i2 << endl << i3 << endl;
  if (i1==i2) cout << "i2 y i3 son exactamente iguales" << endl;
  else cout << "i2 y i3 no son exactamente iguales" << endl;
  Libro::informacion(i1);
  cout << "------" << endl;
  i2=i1;
  cout << i1 << endl << i2 << endl << i3 << endl;
  cout << "------" << endl;
  Libro::setIVA(0.10); //el IVA debe ser como maximo del 21%
  Libro::informacion(i3);
  if (i1==i2)
    cout << "i1 y i2 son exactamente iguales" << endl;
  return 0;
}
```

```
29-12-2024
29/12/2024, 30/10/2001, 15/11/2000
1: La Odisea (Homero)
2: La Odisea (Homero)
d1 y d2 no son exactamente iguales
1: La Odisea (Homero)
1: La Odisea (Homero)
d1 y d2 son exactamente iguales
--LIBROS--
3: Genesis (Pedro Salinas) - 29/12/2024 Ed. Planeta, 10€
4: Perdidos (Juan Rulfo) - 30/10/2001 Ed. Alfaguara, 12.75€
5: Perdidos (Juan Rulfo) - 30/10/2001 Ed. Alfaguara, 12.75€
i2 y i3 no son exactamente iguales
3: Genesis (Pedro Salinas) - 29/12/2024 Ed. Planeta, 10€
precio sin IVA(21%): 8.26446
------
3: Genesis (Pedro Salinas) - 29/12/2024 Ed. Planeta, 10€
3: Genesis (Pedro Salinas) - 29/12/2024 Ed. Planeta, 10€
5: Perdidos (Juan Rulfo) - 30/10/2001 Ed. Alfaguara, 12.75€
------
5: Perdidos (Juan Rulfo) - 30/10/2001 Ed. Alfaguara, 12.75€
precio sin IVA(10%): 11.5909
i1 y i2 son exactamente iguales
```

1. Implementa en C++ la clase Libro y los métodos **estrictamente necesarios** para que el main( ) pueda ejecutarse correctamente y genere la salida indicada (aquellos métodos que no se invocan, como por ejemplo, los métodos get o set no hay que implementarlos).

   Nota: Suponed que los métodos de las clases Fecha y Documento que **estrictamente se usan** en el main() están implementados y funcionan correctamente. **No es posible añadir ningún método adicional a dichas clases.**

SOLUCION:

Para implementar la clase libro tenemos que tener claro qué métodos tienen las clases Fecha y Documento ya que en el enunciado se dice "suponed que los métodos de las clases Fecha y Documento que **estrictamente se usan** en el main() están implementados y funcionan correctamente. **No es posible añadir ningún método adicional a dichas clases".**

Viendo el main() deducimos que los métodos de la clase Fecha y de la clase Documento son **estrictamente** los siguientes:

```
class Fecha {
  int dia, mes, anio;
public:
  Fecha(int d, int m, int a);
  Fecha(const Fecha &f);
  Fecha& operator=(const Fecha& f);
  void setFecha(int d, int m, int a);
  int getDia() const;
  int getMes() const;
  int getAnio() const;
  friend ostream& operator<<(ostream &s,
                             const Fecha &f);
};
```

```
class Documento {
  static int contador;
  int idDocumento;
  char *titulo;
  char *autor;
public:
  Documento (char *t, char *a);
  Documento(const Documento& d);
  Documento& operator=(const Documento& d);
  bool operator==(const Documento &d) const;
  friend ostream& operator<<(ostream &s,
                             const Documento &d);
};
int Documento::contador=1;
```

La clase Fecha tiene un **constructor de copia** y un **operador de asignación**, aunque en el main() no se usan estrictamente: ambos los genera el compilador automáticamente y, al no tener punteros, funcionan correctamente **y los podemos usar**.
Para implementar la clase Libro "No es posible añadir ningún método adicional a las clases Fecha y Documento"

```
class Libro: public Documento { //el constructor Libro debe llamar al constructor Documento (su padre)
static double IVA;              //en la zona de inicializadores y pasarle al padre lo que es suyo
  Fecha fechaEdicion; //los atributos que son objetos hay que ponerlos en la zona de
                      //inicializadores del constructor (uso el constructor de copia de Fecha)
  char *editorial;    //los atributos punteros hay que reservar memoria con new y liberar con delete
  double precio;
public:
  Libro (char *t, char *a, Fecha f, char *e, double p): Documento(t,a), fechaEdicion(f) {
    editorial=new char[strlen(e)+1];
    strcpy(editorial, e);
    precio=p;
  }
  ~ Libro () { delete [] editorial; } //el destructor de Libro libera lo que es estrictamente suyo, de
                                      //liberar loque hereda del padre se ocupa el destructor Documento
  Libro(const Libro& li): Documento(li), fechaEdicion(li.fechaEdicion) {
    editorial=new char[strlen(li.editorial)+1];
    strcpy(editorial, li.editorial);
    precio=li.precio;
  }
//el operator= de Libro clona lo que es suyo y lo que es del padre (Documento) lo clona el padre
  Libro& operator=(const Libro& li) {
    if (this != &li) { //lo primero que hay que ver es si el operador izq y derecho son diferentes
      Documento::operator=(li); //Libro debe llamar al operator= del padre para que clone su parte
      fechaEdicion=li.fechaEdicion; //uso el operator= de Fecha
      delete [] editorial;
      editorial=new char[strlen(li.editorial)+1]; strcpy(editorial, li.editorial);
      precio=li.precio;
    }
    return *this;
  }

  static void informacion(const Libro &li) {
    cout << li << endl << "precio sin IVA(" << Libro::IVA *100 << "%): "
         << li.precio/(1+Libro::IVA) << endl;
  }

  static void setIVA(double i) {
    if (i<=0.21 && i>=0)
      Libro::IVA=i;
  }
//el operator== de Libro compara lo que es suyo y lo que es del padre (Documento) lo compara el padre
  bool operator==(const Libro &li) const {
    bool mismaFecha=fechaEdicion.getDia()==li.fechaEdicion.getDia() && //para comparar fechas debo
                    fechaEdicion.getMes()==li.fechaEdicion.getMes() && //comparar dia, mes y año ya
                    fechaEdicion.getAnio()==li.fechaEdicion.getAnio(); //que Fecha no tiene operator=
    return (Documento::operator==(li) && mismaFecha &&
            strcmp(editorial, li.editorial)==0 && precio==li.precio);
  }
//el operator << de Libro muestra lo que es suyo y lo que es del padre (Documento) lo muestra el padre
  friend ostream& operator<<(ostream &s, const Libro &li) {        //al castear li a Documento hago
    s << (Documento &)li << " - " << li.fechaEdicion << " Ed. "    //que el operator<< que se ejecute
      << li.editorial << ", " << li.precio << "€";                 //sea el de la clase Documento, que
    return s;                                                      //se encarga de mostrar lo que es
  }                                                                //de Documento
};

double Libro::IVA=0.21;
```

2- (3.50 Puntos). Java. Dada la clase Fecha, la clase Documento y su clase hija Libro anterior

1. Implementa el main( ) anterior en lenguaje Java, de manera que conceptualmente haga lo mismo (cree 3 objetos Fecha y los muestre por pantalla, cree 2 objetos Documento y los muestre por pantalla, compare los 2 documentos para ver si son iguales o no, haga que el Documento d2 sea un clon exacto del documento d1, cree 3 Libros y los compare, etc.) **(1.00 punto)**

SOLUCION:

En Java los objetos se crean usando el operador new, se muestran por pantalla con println() e invocando el método toString(), se comparan usando el método equals() (el operador == es para saber si 2 objetos son físicamente el mismo, no para saber si son iguales) y se clonan usando el método clone() (el método clone se hereda de Object y como devuelve un Object siempre hay que convertirlo –**castearlo**- al tipo al que queremos asignarlo).

Por tanto, el main() codificado en Java queda de la siguiente manera:

```
public static void main(String[] args) {
   final Fecha f1=new Fecha(29,12,2024), f2=new Fecha(30,10,2001); //f4=new Fecha(); //no permitido
   Fecha f3=new Fecha(1,1,2000); f3.setFecha(15,11,2000);
   System.out.println(f1.getDia() + "-" + f1.getMes() + "-" + f1.getAnio());
   System.out.println(f1 + ", " + f2 + ", " + f3); //Java internamente invoca el método toString()
// System.out.println(f1.toString() + ", " + f2.toString() + ", " + f3.toString());

   Documento d1=new Documento("La Odisea", "Homero"), d2=new Documento(d1);
   System.out.println(d1 + "\n" + d2); //Java internamente invoca el método toString()
// System.out.println(d1.toString() + "\n" + d2.toString());

   if (d1.equals(d2))
      System.out.println("d1 y d2 son exactamente iguales");
   else
      System.out.println("d1 y d2 no son exactamente iguales");
   d2=(Documento)d1.clone(); //clone() devuelve un Object que hay que castear al tipo Documento
   System.out.println(d1 + "\n" + d2); //Java internamente invoca el método toString()
// System.out.println(d1.toString() + "\n" + d2.toString());

   if (d1.equals(d2))
     System.out.println("d1 y d2 son exactamente iguales");
   System.out.println("--LIBROS--");
   Libro.setIVA(0.27); //el IVA debe ser como maximo del 21%
   Libro i1=new Libro("Genesis", "Pedro Salinas", f1, "Planeta", 10.00);
   Libro i2=new Libro("Perdidos", "Juan Rulfo", f2, "Alfaguara", 12.75);
   Libro i3=new Libro(i2);
   System.out.println(i1 + "\n" + i2 + "\n" + i3); //Java internamente invoca el método toString()
// System.out.println(i1.toString() + "\n" + i2.toString() + "\n" + i3.toString());

   if (i1.equals(i2))
      System.out.println("i2 y i3 son exactamente iguales");
   else
      System.out.println("i2 y i3 no son exactamente iguales");
   Libro.informacion(i1);
   System.out.println("------");
   i2=(Libro)i1.clone(); //clone() devuelve un Object que hay que castear al tipo Libro
   System.out.println(i1 + "\n" + i2 + "\n" + i3); //Java internamente invoca el método toString()
// System.out.println(i1.toString() + "\n" + i2.toString() + "\n" + i3.toString());

   System.out.println("------");
   Libro.setIVA(0.10); //el IVA debe ser como maximo del 21%
   Libro.informacion(i3);
   if (i1.equals(i2))
     System.out.println("i1 y i2 son exactamente iguales");
}
```

2. Implementa en Java la clase Libro siguiendo las mismas indicaciones y restricciones que en la pregunta de C++. **(2.50 puntos)**

SOLUCION:

Para implementar la clase libro tenemos que tener claro qué métodos tienen las clases Fecha y Documento y **No es podemos añadir ningún método adicional a dichas clases.**

Viendo el main() deducimos que los métodos de la clase Fecha y de la clase Documento son **estrictamente** los siguientes:

```
class Fecha { //clase MUTABLE
  private int dia, mes, anio;
  public Fecha(int d, int m, int a) {...}
  public int getDia() {...}
  public int getMes() {...}
  public int getAnio() {...}
  public void setFecha(int d, int m, int a) {...}
//en el main no se usa equals() ni clone() → no
//existe y el heredado de Object no funciona bien
  public String toString() {...}
};
```

```
class Documento implements Cloneable {//INMUTABLE
  private static int contador=1;
  private int idDocumento;
  private String titulo; //INMUTABLE
  private String autor;  //INMUTABLE
  public Documento(String t, String a) {...}
  public Documento(Documento d) {...}
  public Object clone(){ return new Documento(this); }
  public boolean equals(Object o) {...}
  public String toString() {...}
};
```

Java no crea constructor de copia (como C++) ni operador de asignación (Java no permite sobrecargar operadores) y los métodos que heredamos de Object -toString(), equals() y clone()- debemos sobrescribirlos para que funcionen bien.

Antes de codificar la clase Libro debemos realizar la siguiente observación:

Si vemos la salida por pantalla, vemos que al crear un documento a partir de otro `d2=new Documento(d1);` se asigna un `idDocumento` distinto y al clonar con `clone()` el `idDocumento` debe ser el mismo. Por tanto, la implementación del `clone()` de la clase `Documento` no puede ser `return new Documento(this)` ya que entonces el `idDocumento` sería distinto.
Como queremos que sea el mismo, debemos implementar el `clone()` de `Documento` de otra forma, o bien llamando al `clone()` de la clase `Object` (izquierda) o bien creando un "constructor de copia" sobrecargado que asigna el mismo `idDocumento`, el cual es llamado internamente en la implementación del método `clone()` (derecha)

```
class Documento { //clase INMUTABLE
  ...
  public Object clone() {        // captura y trata excepciones
     Documento d=null;           // no las propaga
     try {
        d=(Documento)super.clone();
       //los atributos inmutables no hay que clonarlos, solo los mutables
     } catch(CloneNotSupportedException ex){
        System.out.println(" error al duplicar");
     }
     return d;
  }
};
```

```
class Documento { //clase INMUTABLE
  ...
  public Documento(Documento d) {
    idDocumento=contador;     contador++;
    titulo=d.titulo;          autor=d.autor;
  }
  protected Documento(Documento d, int notUsed) {
    idDocumento=d.idDocumento
    titulo=d.titulo;          autor=d.autor;
  }
  public Object clone() { return new Documento(this,1); }
};
```

A la hora de implementar el `clone()` de la clase `Libro` ocurre lo mismo que en la clase Documento: debemos implementar el `clone()` de `Libro` o bien llamando al `clone()` de la clase `Documento` (izquierda) o bien creando un "constructor de copia" sobrecargado, el cual es llamado internamente en la implementación del método `clone()` (derecha)

```
class Libro { //clase INMUTABLE
  static double IVA;
  private static double IVA=0.21;
  private Fecha fechaEdicion; //MUTABLE
  private String editorial;   //INMUTABLE
  private double precio;
  ...
  public Object clone() {
     Libro li=null;
  //clone de Documento no propaga excepciones (las captura)→ no try catch
     li=(Libro)super.clone();
    //los atributos inmutables no hay que clonarlos, solo los mutables
    //Fecha no tiene clone() → creamos la Fecha con el constructor y new
     li.fechaEdicion=new Fecha(fechaEdicion.getDia(),
        fechaEdicion.getMes(),fechaEdicion.getAnio());
     return li;
  }
};
```

```
class Libro { //clase INMUTABLE
  static double IVA;
  private static double IVA=0.21;
  private Fecha fechaEdicion; //MUTABLE
  private String editorial;   //INMUTABLE
  private double precio;
  ...
  public Libro(Libro li) {
    super(li);
    ...
  }
  protected Libro(Libro li, int notUsed) {
    super(li, notUsed); //llama al sobrecargado de Documento
    ...                 //que deja el mismo idDocumento
  }
  public Object clone() { return new Libro(this,1); }
};
```

La clase `Libro` en Java queda de la siguiente manera:

```
public class Libro extends Documento { //el constructor Libro debe llamar al constructor Documento
  static double IVA;                   //(su padre) con super y pasarle al padre lo que es suyo
  private static double IVA=0.21;
  private Fecha fechaEdicion; //MUTABLE
  private String editorial;   //INMUTABLE
  private double precio;      //TIPO PRIMITIVO (NO ES UN OBJETO)


  public Libro (String t, String a, Fecha f, String e, double p) {
     super(t,a);              //Fecha es MUTABLE (fechaEdicion no debe apuntar al mismo objeto Fecha al que apunta f, sino a una copia)
     fechaEdicion=new Fecha(f.getDia(), f.getMes(), f.getAnio()); //Fecha es MUTABLE
     editorial=e; //String es INMUTABLE (da igual que editorial apunte al mismo String al que apunta e)
     precio=p; //double es TIPO PRIMITIVO
  }


  public Libro(Libro li) {
     super(li);               //Fecha es MUTABLE (fechaEdicion no debe apuntar el mismo objeto que li.fechaEdicion, sino a una copia))
     fechaEdicion=new Fecha(li.fechaEdicion.getDia(), li.fechaEdicion.getMes(), li.fechaEdicion.getAnio());
     editorial=li.editorial; //String es INMUTABLE (da igual que editorial y li.editorial apunten al mismo objeto String)
     precio=li.precio; //double es TIPO PRIMITIVO
  }

//el método clone de Libro no puede ser return new Libro(this) PORQUE asignaría un idDocumento distinto
  public Object clone() { //como queremos que idDocumento sea el mismo llamamos al clone de Documento
     Libro li=null;
  //clone de Documento no propaga excepciones (las captura) → no try catch
     li=(Libro)super.clone();
  //los atributos inmutables (editorial) no hay que clonarlos, solo los mutables (fechaEdicion)
  //Fecha no tiene clone() → creamos la Fecha con el constructor y new
     li.fechaEdicion=new Fecha(fechaEdicion.getDia(), fechaEdicion.getMes(),fechaEdicion.getAnio());
     return li;
  }


//el equals() de Libro compara lo que es suyo y lo que es del padre (Documento) lo compara el padre
  public boolean equals(Object o) {
     if (this.getClass()!=o.getClass()) //son distinta clase
       return false;
     Libro li=(Libro) o; //o es de tipo Object y hay que castearlo al tipo Libro
     boolean mismaFecha=fechaEdicion.getDia()==li.fechaEdicion.getDia() && //para comparar fechas debo
                    fechaEdicion.getMes()==li.fechaEdicion.getMes() && //comparar dia, mes y año ya
                    fechaEdicion.getAnio()==li.fechaEdicion.getAnio(); //que Fecha no tiene equals()
     return (super.equals(li) && mismaFecha && editorial.equals(li.editorial) && precio==li.precio);
  }


  public static void informacion(Libro li) {
     System.out.println (li + "\nprecio sin IVA(" + IVA*100 + "%): " + li.precio/(1+IVA));
  // System.out.println (li.toString() + "\nprecio sin IVA(" + IVA*100 + "%): " + li.precio/(1+IVA));
  }


  public static void setIVA(double i) { if (i<=0.21 && i>=0)  IVA=i;  }


//toString() de Libro devuelve lo que es suyo y lo que es del padre (Documento) lo devuelve el padre
  public String toString() {
     String s= super.toString() + " - " + fechaEdicion + " Ed. " + editorial + ", " + precio + "€";
     return s;
  }
}
```

El `clone()` de la clase `Libro` podemos codificarlo de otra manera usando un "constructor de copia" sobrecargado:

```
public class Libro extends Documento { //clase INMUTABLE
  ...
  public Libro(Libro li) {
    super(li);
    ...
  }
  protected Libro(Libro li, int notUsed) {
    super(li, notUsed); //llama al sobrecargado de Documento
    ...                 //que deja el mismo idDocumento
  }
  public Object clone() { return new Libro(this,1); }
};
```

3- (2.00 Puntos). Excepciones.

1. **(C++)** Dado el siguiente código:

```
#include <iostream>
#define N 5

using namespace std;

class ENoValido: public exception {
    const char *mensaje;
public:
    ENoValido(const char *m="ERROR")
        : mensaje( m ) { }
    const char *what() const throw(){
        return mensaje;
    }
};

int main() {
    if (N<4)
      set_terminate( terminar );
    try{
        proceso(N);
    }catch(ENoValido &e ){
        cout << "CUIDADO: " << e.what() << endl;
    }catch( exception &e ){
        cout << "AVISO: " << e.what() << endl;
    }catch( ... ){
        cout << "Error imprevisto\n";
    }

    cout << "CONTINUAR\n";
    calculo(N-1);
    cout << "FINAL CORRECTO\n"; return 0;
}
```

```
void terminar( ) { //funcion void
    cout << "FINAL ANORMAL!!\n";
    system("PAUSE");    exit(EXIT_SUCCESS);
}

void calculo(int n) throw (exception, ENoValido) {
  if (n==3)
    throw ENoValido("CERO");
  else if (n==1)
    throw ENoValido();
  else if (n==0)
    throw exception();
  cout << "Numero:" << n << endl;
}

void proceso(int valor) {
    try {
      switch(valor) {
        case 0: calculo (1); break;
        case 2: calculo (2);
                throw float();
                break;
        case 5: calculo (8);
        default:cout << "SEGUIR\n";
       }
    }catch ( exception &e ){
        cout << e.what() << endl;
        if (valor < 2)
            throw;
    } catch( ... ){
        cout << "ERROR NO IDENTIFICADO\n";
    }
}
```

Indica la salida que se genera por pantalla cuando N vale 0, 1, 2 y 5: **(1.00 puntos)**

SOLUCION:

0	ERROR	CUIDADO: ERROR	CONTINUAR	Numero:-1	FINAL CORRECTO
1	SEGUIR	CONTINUAR	FINAL ANORMAL!!		
2	Numero:2	ERROR NO IDENTIFICADO	CONTINUAR	FINAL ANORMAL!!	
5	Numero:8	SEGUIR	CONTINUAR	Numero:4	FINAL CORRECTO

2. **(JAVA)** Dado el siguiente código:

```
class ENoValido extends Exception {
  public ENoValido(String mensaje) { super(mensaje); }
  public ENoValido() { super(); }

  public String toString() {
    String s="Error ";
    s=s+ getClass().getName()+ ": " + getMessage();
    return s;
  }
};

public class Principal {
  public static void main(String [] args) throws
ENoValido, Exception {
    try{
        proceso(N);
    }catch( ENoValido e ){
        System.out.println("OJO!! ");
    }catch( Exception e ){
        System.out.println("CUIDADO!! ");
    }finally {
      System.out.println(N);
    }

    System.out.println("CONTINUAR");
    System.out.println(calculo(N));
  }
```

```
 public static int calculo(int n) throws Exception,
ENoValido {
    if (n==0)
      throw new ENoValido("CERO");
    else if (n==1)
      throw new Exception("ABORTAR");
    else
      System.out.println("Numero:" + n);
    return n/2;
  }

  public static void proceso(int v) throws Exception {
    try {
      switch(v) {
        case 1: calculo (1); break;
        case 2: calculo (2);
        case 5: throw new Exception();
        default: System.out.println(calculo(v));
       }
    }catch (ENoValido e) {
        System.out.println(e);
    } catch(Exception e){
        System.out.println("ERROR RARO");
        throw e;
    }
  }
}
```

Indica la salida que se genera por pantalla cuando N vale 0, 1, 2 y 5: **(1.00 puntos)**

SOLUCION:

```
0  Error ENoValido: CERO     0     CONTINUAR    (el programa aborta con mensaje) Error ENoValido: CERO
1  ERROR RARO    CUIDADO!!   1     CONTINUAR    (el programa aborta con mensaje) java.lang.Exception: ABORTAR
2  Numero:2      ERROR RARO        CUIDADO!!     2            CONTINUAR         Numero:2          1
5  ERROR RARO    CUIDADO!!   5     CONTINUAR     Numero:5     2
```

3- (1.00 Puntos). Corregir Errores / Justificar Respuestas.

1) **(C++)** Indica los posibles errores del siguiente código justificando el motivo: **(0.50 puntos)**

```
#include <iostream>
#include <cstring>
using namespace std;

class A {
  static int contador=0;
  const int a;
  char *cad;
public:
  A(char* c) {
    a=contador++;
    strcpy(cad, c);
  }
  static void ver() const { cout << a << "," << cad << endl; }
  int getAsiguiente() const { return a++; }
};
```

SOLUCION:

Error 1: el atributo estático **contador** se debe inicializar fuera de la clase

```
class A {
  static int contador;
  ...
};

int A::contador=0;
```

Error 2: los métodos estáticos sólo pueden acceder a atributos estáticos

```
~~static~~ void ver() const { cout << a << "," << cad << endl; }
```

Error 3: el atributo constante **a** debe iniciaizarse en la zona de inicializadores del constructor.

```
  A(char* c): a(contador++) {
    strcpy(cad, c);
  }
```

Error 4: falta reservar memoria para el puntero **cad** antes de copiar con strcpy

```
  cad=new char[strlen(cad)+1];
  strcpy(cad, c);
```

Error 5: el atributo **a** es constante y no se puede incrementar.

```
  int getAsiguiente() const { return a+1; }
```

2) **(JAVA)** Al ejecutar el main( ), indica lo que se ve en pantalla. Justifica la respuesta. **(0.50 puntos)**

```
public class Contador {
  private int n;

  public Contador(int v) { n = n+v; }

  public String toString() {
    String s = "valor = " + n;
    return s;
  }
```

```
  public static void setContador(Contador c, int i) {
    c = new Contador(i);
  }
  public static void main(String[] args) {
    Contador c = new Contador(8);
    Contador.setContador(c, 15);
    System.out.println(c);
  }
}
```

SOLUCION: por pantalla se muestra “valor = 8”

En el main() c apunta a un objeto Contador con valor 8 y al método setContador le pasamos una copia del puntero c. Esa copia del puntero c hacemos que apunte a un objeto Contador con valor 15, pero el c original sigue apuntando al objeto Contador con valor 8.